

GiroCerto — Análise de Mercado e Oportunidades do Torre

Comparação entre o GiroCerto hoje (schema + mockups) e o mercado de despacho de motoboy/última milha pra loja local no Brasil, em 16/08/2026, seguida de um levantamento específico do que o projeto Torre (fleet-orchestrator — orquestração de frota de drones agrícolas, mesmo desenvolvedor) já resolveu e é transferível.

Nota metodológica: as afirmações sobre o GiroCerto vêm de leitura direta do

`db/schema.sql` (807 linhas) e dos 3 mockups HTML (`cadastro-loja.html` , `painel-loja.html` , `app-entregador.html`) nesta sessão — inclusive checagem específica de que nenhum dos mockups chama `navigator.geolocation` em nenhum lugar (confirmado por grep). As afirmações sobre o mercado vêm de pesquisa web ativa feita nesta sessão, com fonte para cada achado (ver seção de fontes ao final). As afirmações sobre o Torre vêm de conhecimento direto do código desse projeto irmão.

(a) Onde o GiroCerto já está à frente do mercado

O schema é notavelmente mais maduro do que os 3 mockups sozinhos sugerem — vários pontos aqui não têm paralelo claro nos concorrentes pesquisados.

1. RLS implementada de verdade, policy por policy — não só habilitada. Toda

tabela sensível (`pedidos` , `entregadores` , `localizacoes_entregador` , `alertas_seguranca` etc.) tem `ENABLE ROW LEVEL SECURITY` e políticas reais escopando por `tenant_id` / `entregador_id` via `auth.uid()` . Isso é uma postura de segurança madura desde o dia 1 — nenhum dos concorrentes pesquisados documenta publicamente esse nível de detalhe de arquitetura, então não dá pra comparar diretamente, mas é uma prática correta rara de se ver num MVP.

2. Sistema de fadiga do entregador — alerta + bloqueio de descanso obrigatório.

`horas_alerta_fadiga` , `horas_descanso_obrigatorio` , `entregadores.bloqueado_ate` .

Nenhum concorrente pesquisado tem isso. E o momento não podia ser mais oportuno: a

NR-1 (vigente desde jan/2025) exige que empresas incluam fatores de risco psicossocial (jornada exaustiva, sobrecarga) no Programa de Gerenciamento de Riscos, e um estudo recente aponta que afastamentos por fadiga/esgotamento ****triplicaram** entre 2023 e 2025**. Isso não é feature de nice-to-have — é o tipo de coisa que vira argumento de compliance numa fiscalização.

3. "Selo Entrega Justa" — avaliação bidirecional loja↔motoboy, calculada, anti-manipulação.

View `selo_entrega_justa` : só fica ativo com infraestrutura declarada (banheiro/abrigo)

e nota real média ≥ 4.0 e volume mínimo de 10 avaliações nos últimos 30 dias —

não dá pra forjar com 2 notas boas. Pesquisei especificamente por isso: o que existe no mercado é selo de "documentos verificados" (unidirecional, cliente avalia fornecedor). Não encontrei nenhum concorrente com avaliação do motoboy sobre a loja virando selo público. É um diferencial de marca genuíno, não só de produto.

4. Failover automático de despacho com repique e timeout. `tentativas_despacho` + `segundos_repique_notificacao` / `segundos_timeout_despacho` no tenant — chama o próximo entregador disponível sozinho se o primeiro não responder. Foody Delivery menciona "despacho automático" mas não documenta a lógica; nenhum concorrente pesquisado detalha esse mecanismo publicamente.

5. Segurança com confirmação humana antes de escalar. `alertas_seguranca` nunca aciona 190 sozinho — fluxo é `aguardando_confirmacao` → `confirmado_ok` **ou** `escalado_loja` → só então (se necessário) `acionado_190`. Isso importa mais do que parece: a pesquisa encontrou um aumento real de golpes de "falso motoboy" e roubo de motos de entregador (ex: +106,9% em ocorrências com um modelo específico no 2º trimestre de 2026) — um sistema que aciona polícia automaticamente por qualquer desvio de GPS geraria alarme falso o tempo todo; a confirmação humana evita isso sem abrir mão da detecção.

6. Modelo de vínculo flexível (fixo vs. freelance). `tipo_vinculo`, `valor_fixo`, `periodicidade_fixo` — a maioria dos concorrentes pesquisados (Wappa, Lalamove, Loggi Expresso) são modelo de frete avulso puro. O GiroCerto atende os dois mundos: loja com motoboy fixo mensal/diário E freelance por entrega.

7. Compensação de espera excedente — ataca a dor central do produto.

`tempo_espera_tolerado_min` + `valor_por_minuto_espera_excedente`. Nenhum concorrente pesquisado trata "tempo parado esperando o pedido ficar pronto" como item de remuneração explícito — é exatamente o "ciclo ocioso" que dá nome ao produto, e é tratado como parâmetro de primeira classe, não afterthought.

(b) Gaps que importam — priorizados

● Crítico

1. Rastreamento ao vivo — a estrutura existe, a captura não.

`localizacoes_entregador` está pronta no schema (lat/lng/timestamp/rota_id), mas **nenhum dos 3 mockups chama `navigator.geolocation` em nenhum lugar** — confirmei via grep, não é suposição. A tabela nunca recebe uma linha na prática hoje. Isso também significa que a detecção de desvio de rota (`alertas_seguranca`) e o SOS não têm dado nenhum pra funcionar de verdade ainda — é a base de tudo que falta. Mercado: Foody Delivery, Loggi Expresso e Motoboy.App tratam rastreamento ao vivo (inclusive link público pro cliente final) como recurso básico, não diferencial.

→ Ver seção Torre abaixo — é literalmente o mesmo problema que o Torre já resolveu pra telemetria de drone.

2. Detecção de "motoboy parado" não existe — só desvio de rota e SOS manual.

`alertas_seguranca.tipo` só aceita `'desvio_rota'` ou `'sos_manual'`. Uma vez que a captura de posição existir (gap #1), o dado pra detectar "motoboy parado sem se mexer" (bateria de celular acabou, acidente, assalto em andamento) já está ali — falta só a lógica. É um alerta de segurança genuinamente importante que hoje tem zero cobertura.

3. Roteirização multi-parada é estrutural, não otimizada. `ordem_na_rota` existe e o app do entregador já mostra "parada 1, 2, 3..." — mas não há nenhum algoritmo de otimização de distância visível no código; a ordem parece sequencial/manual. Foody Delivery e Entregador Online colocam "melhor sequência de paradas" como recurso de vitrine. Pra um produto cuja missão é reduzir km rodado à toa, isso é central, não periférico.

● Importante

4. Sem link público de rastreo pro cliente final. Um comentário no schema (`avaliacao_entrega ... coletada pelo link de rastreo`) sugere que a intenção sempre existiu, mas não há token/rota dedicada implementada — e mesmo que existisse, não teria posição ao vivo pra mostrar (depende do gap #1). Reduz `tentativas_contato` na prática (cliente que vê "motoboy a 5 min" liga menos perguntando "cadê meu pedido").

5. Sem seguro/indenização do motoboy. A **Lei 14.297/2022** já obriga apps de entrega a contratarem seguro de vida/acidentes pro entregador — isso não é só feature de mercado, é exposição legal/compliance. Foody Delivery já anuncia integração facilitada com seguradora.

6. Sem chat in-app. Contato hoje é só `tentativas_contato.tipo = 'ligacao'` (fora do sistema, sem registro do conteúdo). Chat interno cria trilha auditável e reduz fricção — Foody Delivery e VEX MENU (esse último com transcrição de áudio por IA) já oferecem.

7. "Relatórios" existe no menu, mas sem função de agregação implementada.

`painel-loja.html` tem a aba, mas não há `carregarRelatorios()` / gráfico nenhum no código — é só o link de navegação. Concorrentes tratam dashboard analítico como padrão do produto, não add-on.

8. Precificação estática — sem ajuste por demanda/horário de pico. Tarifa é fixa por tenant (`tarifa_minima` , `valor_por_km_adicional`). Machine Conecta já oferece motor de precificação configurável por faixa/horário. Não é urgente pro MVP, mas é aposta de médio prazo — cuidado extra: dado que "espera não paga" é dor central do produto, um motor de precificação dinâmica mal calibrado pode

contradizer a promessa de previsibilidade que hoje é diferencial.

● Nice-to-have / observações

9. ETA com IA — tendência real (iFood já usa pra prever atraso e reordenar por urgência), mas prematuro: sem rastreamento ao vivo (gap #1), não há dado pra alimentar previsão nenhuma ainda.

10. Gamificação/ranking de entregador — NÃO recomendo copiar. É prática comum em apps grandes, mas a literatura (Metrópoles, artigo acadêmico da Revista APS) trata como mecanismo questionável de controle algorítmico do trabalho. Copiar isso entraria em contradição direta com o próprio posicionamento do GiroCerto (fadiga, Selo Entrega Justa) — o produto já está do lado certo dessa discussão, vale manter.

11. Ciclomotor/e-bike sem campo de placa. A partir de 1º/jan/2026, a regulamentação do Contran passa a exigir Renavam/emplacamento pra ciclomotores até 50km/h e 4kW — hoje só `tipo_veiculo = 'moto'` tem `placa / crlv_validade` no schema; `bicicleta` não tem equivalente. Vale monitorar se isso vira exigência prática antes de precisar de migração.

Cruzamento com o Torre — o que já está resolvido e é transferível

O Torre (fleet-orchestrator) resolveu, pra frota de drones agrícolas, praticamente o mesmo problema estrutural do gap #1 do GiroCerto: ****muitos agentes móveis reportando posição periodicamente, com alerta em tempo real sobre o que sai do esperado.**** Os padrões abaixo não são cópia de código (domínios diferentes), mas a arquitetura é diretamente adaptável.

1. Rastreamento ao vivo (resolve gap crítico #1). Torre usa WebSocket (`telemetryHub.js / useTelemetrySocket.js`) com fallback de polling HTTP, e um buffer de escrita em lote (`telemetryBuffer.js`) que agrupa inserções antes de gravar no banco — evita 1 INSERT por segundo por entregador ativo. Pro GiroCerto: o app do entregador chamaria `navigator.geolocation.watchPosition` e mandaria pro mesmo tipo de endpoint bufferizado; o painel da loja assinaria WebSocket (com polling como rede de segurança, exatamente como o Torre já faz).

****2. Alerta de "motoboy parado" (resolve gap crítico #2) — é literalmente o mesmo algoritmo.**** O Torre acabou de ganhar `computeStalledSeconds()` : anda de trás pra frente pelo histórico de posição enquanto ela for a mesma (com tolerância pra ruído de GPS) e devolve há quanto tempo esse "platô" dura; dispara alerta só se o agente estiver em status ativo (`in_progress` , não pausado/retornando). Pro GiroCerto: mesma lógica, mesma tabela de dados (`localizacoes_entregador` já tem timestamp+posição), só troca "drone parado durante missão" por "motoboy parado

durante rota" (`rotas_entrega.status = 'em_entrega'` faz o papel do `in_progress`).

****3. Geofence/desvio de rota com tolerância a ruído** — o GiroCerto já pensou nisso, o Torre validou a mesma ideia em produção.****** O Torre só dispara alerta de "fora da área" depois de 2+ leituras consecutivas fora do polígono (`ST_Contains`), pra não alarmar por 1 ponto de GPS ruim na borda. O `km_desvio_alerta` do GiroCerto já existe como conceito — vale garantir que a implementação real também exija confirmação por mais de 1 leitura antes de criar o `alertas_seguranca` , não só a primeira leitura fora da polyline.

4. Lista mestre com badge de alerta sempre visível, mesmo sem foco. O Torre acabou de resolver exatamente esse problema pra "acompanhar 2+ missões simultâneas": lista de cards sempre visível, cada um mostrando alerta/bateria/progresso ao vivo mesmo sem estar expandido — resolve o risco de "só percebo o problema se estiver olhando pra aquela missão especificamente". A aba "Rotas ativas" do `painel-loja.html` tem exatamente esse mesmo risco hoje (lista simples, sem indicador vivo por rota) — o mesmo padrão de card com badge se aplica direto, sobretudo pros dois alertas de segurança (SOS, desvio).

5. Rastro real percorrido no mapa. O Torre desenha o trajeto real do drone como polyline no mapa (não só a posição atual), reconstruído a partir do histórico de telemetria. `localizacoes_entregador` já grava esse histórico — o mesmo padrão (buscar histórico + desenhar linha + acumular pontos novos ao vivo) se aplica ao mapa do painel da loja pra mostrar o trajeto de uma rota em andamento, não só o ponto atual.

6. Auditoria de ações críticas de segurança. O Torre tem tabelas de auditoria dedicadas pra ações sensíveis (quem confirmou, quando, por quê) em decisões de segurança de voo. `alertas_seguranca` tem status mas não tem coluna de "quem confirmou"/"quem escalou" — vale considerar `confirmado_por / escalado_por` (referenciando `usuarios_loja`) seguindo o mesmo princípio: uma decisão de segurança precisa de rastro de quem tomou ela.

7. Via de mão dupla — o GiroCerto já fez melhor que o Torre num ponto. O Torre teve como achado crítico #1 da sua própria auditoria de segurança "RLS habilitada sem nenhuma policy" — proteção que parecia existir mas não existia de fato. O GiroCerto fez isso certo desde o schema inicial (policy por policy, de verdade). Não é um ponto de aprendizado do Torre pro GiroCerto — é o contrário.

8. Convenção de responsividade por papel, quando o app virar código real. O Torre estabeleceu a regra "tela do operador só precisa de tablet/PC, tela do proprietário precisa também de celular". Pro GiroCerto, a lógica provavelmente inverte parcialmente: o app do entregador já é mobile-first por natureza, mas o `painel-loja.html` (usado por dono E funcionário de balcão) tem uma chance real de precisar ser mobile desde o início — dono de loja pequena checando o painel pelo

próprio celular é um cenário bem mais provável nesse nicho do que no caso do Torre.

Lista priorizada de ação (top 6)

#	Item	Esforço estimado
1	Capturar posição real (<code>navigator.geolocation.watchPosition</code>) no app do entregador	Médio
2	Endpoint bufferizado + WebSocket no painel da loja (padrão <code>telemetryBuffer.js</code> / <code>telemetryHub.js</code> do Torre)	Médio-Alto
3	Alerta de "motoboy parado" reaproveitando <code>computeStalledSeconds()</code> do Torre	Baixo (uma vez que #1/#2 existam)
4	Exigir 2+ leituras fora da rota antes de criar <code>alertas_seguranca</code> de desvio (evitar falso positivo)	Baixo
5	Implementar <code>carregarRelatorios()</code> — a aba já existe no menu, falta a função	Baixo-Médio
6	Algoritmo de otimização de ordem das paradas (nearest-neighbor é suficiente pra V1, não precisa ser TSP ótimo)	Médio

Os itens 1-4 são a base de tudo — sem captura de posição real, nenhum dos outros alertas de segurança tem dado pra funcionar, e é justamente onde o Torre já tem arquitetura validada em produção pra reaproveitar.

Fontes consultadas

Concorrentes: Foody Delivery, Foody Delivery — restaurantes com frota própria, InstaDelivery, Entregador Online, SyLog GR EXP, Textus WTRANSP, Sischef — módulo de entregas, Saipos — motoboy, Machine Conecta (via 55content), Wappa (MobileTime), Loggi Expresso, Lalamove São Paulo, Mottu (Exame), iFood-Mottu, oHub — serviços de entrega/motoboy

Features/tendências: Foody Delivery — monitoramento de motoboys, Foody Delivery — sistema de gestão, Foody Delivery — seguro pra entregadores, VEX MENU, DataOcean — precificação dinâmica, E-Commerce Brasil — IA na entrega, iFood — IA no delivery, Metrôpoles — gamificação de entregador, Revista APS — controle algorítmico

Regulamentação/tendências setoriais: Câmara dos Deputados — PL 1184/2025, iFood — regulamentação, Motorshow — Contran ciclomotores 2026, SaúdeBusiness — saúde mental, Motonline — golpe do falso motoboy, ParanáTrack — motos mais roubadas 2026, Guia E-bike — ROI 2026, TudoDeMotos — motos elétricas 2025, Brazil Economy — afastamentos por fadiga